

Metric Dimension Approximation

Algorithm Technical Documentation

Random Geometric Graph Heuristics

Purpose	Approximate the metric dimension of Random Geometric Graphs (RGGs) using three heuristic algorithms and compare results against the ICH baseline.
Language	Python 3 — dependencies: networkx, igraph, shapely, jax, numpy
Baseline	ICH algorithm (multilateration.ich) — exact or near-exact resolving set reference
Key concept	A resolving set S is a set of nodes such that every node in the graph has a unique vector of distances to the nodes in S . The metric dimension is the size of the smallest resolving set.

Table of Contents

1. Shared Infrastructure

All three algorithms share a common set of helper functions and data structures. These are defined once and reused across all methods.

1.1 Graph Representation

Each algorithm accepts a NetworkX RGG (G) and immediately converts it to an igraph object for fast distance computation:

```
g = ig.Graph.from_networkx(G)
dist_matrix = g.distances() # full n×n shortest-path matrix
```

Node positions (coordinates in $[0,1]^2$) are stored as a vertex attribute:

```
pos = g.vs["pos"] # list of (x, y) tuples, one per node
```

1.2 Core Helper Functions

euclidean_distance(p, q)

Computes the Euclidean distance between two 2D coordinate tuples. Used to find which real nodes are closest to heuristic probe points.

```
math.sqrt((p[0]-q[0])**2 + (p[1]-q[1])**2)
```

entropy_of_landmark_candidate(nodes, candidate, dist_matrix)

Measures how informative a candidate node would be as a landmark. For each candidate, it builds a frequency distribution of distances from the candidate to all remaining nodes, then computes Shannon entropy $H = -\sum(p * \log_2(p))$. A higher entropy means the candidate distinguishes nodes more evenly — it is a better landmark.

With $k_{\text{nearest}}=1$ this function acts as a tiebreaker only. With $k_{\text{nearest}}>1$ it is the primary selection criterion within the candidate pool.

checkIfResolvingSet_igraph(g, S, dist_matrix)

Verifies that a set S is a valid resolving set. For every node v in the graph, it computes a distance representation — a tuple of distances from v to each node in S. If all n representations are distinct, S resolves the graph and the function returns True. Returns False immediately on the first collision.

Note: accesses dist_matrix as dist_matrix[s][v] (landmark-indexed rows). This is equivalent to dist_matrix[v][s] for undirected graphs but would differ for directed graphs.

1.3 Zobrist Pruning

After each algorithm builds a resolving set, a post-processing pass removes redundant landmarks using Zobrist hashing. This is the only pruning step — local and incremental pruning were removed after analysis showed they added complexity without meaningful benefit.

How it works

1. Assign each (landmark, distance-value) pair a random 64-bit integer.
2. Hash each node v as the XOR of the random integers for all its (landmark, distance) pairs. If all hashes are distinct, the set is resolving.
3. For each landmark l , tentatively remove it by XOR-ing out its contribution. If the resulting hashes are still all distinct, l is redundant and is permanently removed.
4. Repeat until no further landmarks can be removed.

Functions involved

- `build_zobrist_tables` — allocates random integer table indexed by `[landmark_index][distance_code]`
- `compute_initial_hashes` — builds the initial per-node hash array
- `try_remove_landmark` — tests removal of one landmark without modifying state; returns new hash array if removal is safe, else `None`
- `prune_resolving_set_zobrist_fast` — outer loop that calls the above until convergence

`INF_CODE` is set to `max_finite_distance + 1` so disconnected node pairs (infinite distance) get a consistent hash code rather than crashing the table lookup.

2. The UnitSquareNew Heuristic

The UnitSquareNew class provides the geometric probe-point generator used by both the Grid Method and the Zoomed Square Method. It encodes empirically derived knowledge about where to place landmarks to maximally separate two random points inside a square.

2.1 Motivation

For a unit square centred at the origin, the probability that a randomly placed landmark distinguishes two random points depends on where the landmark is placed. Basin-hopping stochastic optimisation was used to find the sequence of ideal placement positions. The result was two parabolic curves across the square:

```
Curve 1:  y = -0.028964*(x-cx)2 + 0.985804*(x-cx) + 0.630070 + cy
Curve 2:  y =  0.021721*(x-cx)2 + 0.981117*(x-cx) - 0.626609 + cy
```

Curve 1 runs through the upper half of the square (positive y-offset from centre = +0.630070). Curve 2 runs through the lower half (offset = -0.626609). Together they represent the diagonal pattern visible in pattern.pdf — two near-parallel sweeping lines that between them cover the square's area.

2.2 point_generator(cx, cy, r, d)

A Python generator that yields (x, y) probe coordinates for a cell centred at (cx, cy) with half-side r, advancing in steps of d along the x-axis.

Yield order (the pattern from pattern.pdf)

5. Centre points first: evaluate both curves at $x=cx$. Yield the curve 1 point (above centre) then the curve 2 point (below centre), provided each falls inside the cell square $[cx\pm r] \times [cy\pm r]$.
6. Symmetric expansion: step outward to $x = cx+d$ and $x = cx-d$ simultaneously. For each x value yield curve 1 then curve 2 if in bounds. Increment the step counter.
7. Termination: stop when an entire expansion step produces zero in-bounds points on either curve in either direction.

Boundary check

A point (x, y) is accepted only if it lies inside the axis-aligned bounding square of the cell:

```
(cx - r) <= x <= (cx + r)  AND  (cy - r) <= y <= (cy + r)
```

This replaced the original circle check $(dx^2 + dy^2) \leq r^2$, making it consistent with the grid cell geometry which is square, not circular.

Algorithm 1 — Grid Method

get_metric_dimension_of_graph_with_pruning_igraph_grid

3.1 Overview

Entry point	<code>get_metric_dimension_of_graph_with_pruning_igraph_grid(G, r, k_nearest=1)</code>
Inner function	<code>get_metric_dimension_of_unit_square(g, grid, i, j, m, n, r, d, dist_matrix, nodes_set, k_nearest)</code>
Heuristic	UnitSquareNew — two parabolic curves on a unit square
Neighbourhood	Geometric: nodes physically located inside a grid cell
Traversal order	Row-major (top-left to bottom-right)
k_nearest	Default 1 — heuristic alone determines picks; entropy acts as tiebreaker only

3.2 Core Idea

The RGG's $[0,1]^2$ space is partitioned into a uniform grid. Each cell is processed independently: the UnitSquareNew generator proposes probe points within the cell, the closest real node to each probe is selected as a landmark, and the process continues until all nodes in the cell can be distinguished by the collected landmarks. Results from all cells are merged and globally pruned.

The grid cell side length is $2r$ — the same scale as the RGG connection radius — so nodes inside a cell are likely to be mutually connected, and the square heuristic (which was optimised for a unit square at this scale) is geometrically appropriate.

3.3 Step-by-Step

1

Grid construction

Divide $[0,1]^2$ into $m \times n$ cells where $m = n = \text{ceil}(1 / 2r)$. Each node is placed into the cell containing its (x, y) coordinate. Step size $d = 1 / \max(m, n) = 2r$.

```
m = n = math.ceil(1 / (2*r))
d = 1 / max(m, n)
grid = generate_rgg_with_grid(g, m, n)
```

2

Outer loop — iterate cells

Traverse cells in row-major order (row $i=0..m-1$, col $j=0..n-1$). Before each cell, check whether the current `resolving_set` already resolves the whole graph — if so, exit early.

```
for i in range(m):
    for j in range(n):
        if checkIfResolvingSet_igraph(g, resolving_set, dist_matrix): break
```

3	Inner loop — select landmarks for cell Call <code>get_metric_dimension_of_unit_square</code> for the current cell. This function: retrieves nodes in the cell, creates a <code>UnitSquareNew</code> generator for the cell centre, and enters the landmark selection loop. <pre>temp_set, _, nodes_set = get_metric_dimension_of_unit_square(g, grid, i, j, m, n, r, d, dist_matrix, nodes_set, k_nearest)</pre>
4	Probe point → nearest node → best candidate Pull the next ideal point from the generator. Sort remaining nodes by Euclidean distance to the probe; take the top <code>k_nearest</code>. From those, select the node with the highest entropy score as the landmark. <pre>ideal_point = next(gen, None) candidates = sorted(nodes_remaining, key=lambda v: euclidean_distance(pos[v], ideal_point))[:k_nearest] best = max(candidates, key=lambda c: entropy_of_landmark_candidate(...))</pre>
5	Fallback when generator exhausted If the generator yields <code>None</code> before all nodes are resolved, fall back to selecting the highest-entropy node globally among all remaining nodes in the cell. This guarantees termination. <pre>if ideal_point is None: best = max(nodes_remaining, key=lambda c: entropy_of_landmark_candidate(...))</pre>
6	Early exit check After each landmark is added, check if remaining nodes in the cell all have distinct distance signatures to the current local resolving set. If so, break — no more landmarks are needed for this cell. <pre>signatures = {v: tuple(dist_matrix[v][l] for l in resolving_set) for v in nodes_remaining} if len(signatures) == len(set(signatures.values())): break</pre>
7	Merge and global prune After all cells are processed, merge all local resolving sets and run <code>prune_resolving_set_zobrist_fast</code> to remove any globally redundant landmarks. <pre>resolving_set.update(temp_set) resolving_set = prune_resolving_set_zobrist_fast(g, resolving_set, dist_matrix)</pre>

Algorithm 2 — Zoomed Circle Method

get_metric_dimension_of_graph_with_pruning_igraph_circle

4.1 Overview

Entry point	<code>get_metric_dimension_of_graph_with_pruning_igraph_circle(G, r, k_nearest=1, max_iters=1000)</code>
Inner function	<code>get_metric_dimension_of_unit_circle_igraph_new(g, r, temp_center, dist_matrix, node_set, k_nearest)</code>
Heuristic	UnitSquare + UnitCircle — rectangle-bisection on unit square, mapped to a circle of radius r
Neighbourhood	Graph distance $d=1$ (direct neighbours of <code>temp_center</code> , equivalent to Euclidean distance $\leq r$ in an RGG)
Centre selection	Degree-weighted random sampling from unvisited nodes
<code>k_nearest</code>	Default 1

4.2 Core Idea

Rather than imposing a fixed spatial grid, this method adaptively selects a centre node each iteration (biased toward high-degree nodes), then resolves the neighbourhood around that centre. The heuristic is based on the UnitSquare class, which maintains a rectangle-bisection state and at each step returns the midpoint of the largest remaining rectangle. The UnitCircle wrapper maps these unit-square coordinates to the actual circle of radius r around the centre node.

Because centre selection is degree-weighted, the algorithm naturally prioritises dense, well-connected regions of the graph first, which tend to be the hardest to resolve.

4.3 UnitSquare and UnitCircle Classes

UnitSquare

Maintains a division of $[0,1]^2$ into rectangles, stored in a max-heap by area. At each call to `add()`:

8. Pop the largest rectangle from the heap.
9. Determine cut direction: vertical if width > height, horizontal otherwise.
10. Bisect the rectangle at its midpoint along the cut direction.
11. Return the cut coordinate and direction (`is_horizontal` flag).

This guarantees that probe points are placed to maximally subdivide the remaining unresolved area of the square — the greedy rectangle packing optimum.

UnitCircle

Wraps UnitSquare and maps its output coordinates to the circle of radius r around $(\text{center_x}, \text{center_y})$. The mapping:

- If the cut is vertical (not `is_horizontal`): $\text{new_x} = \text{center_x} + \text{coordinate} * 2r$ (or $(1 - \text{coordinate}) * 2r$ if $\text{coordinate} > 0.5$), $y = \text{center_y}$
- If the cut is horizontal: $x = \text{center_x}$, $\text{new_y} = \text{center_y} + \text{coordinate} * 2r$ (or $(1 - \text{coordinate}) * 2r$)

This maps the $[0, 1]$ cut coordinate to a point on the horizontal or vertical diameter of the circle, ensuring probe points stay within the circle.

4.4 Step-by-Step

1	Select centre node From the set of unvisited nodes, select a centre node by degree-weighted random sampling. Nodes with more connections are more likely to be chosen, concentrating effort on dense regions first. <pre>degrees = [g.degree(n) for n in nodes_set] temp_center = random.choices(list(nodes_set), weights=degrees, k=1)[0]</pre>
2	Collect neighbourhood Find all nodes at graph distance exactly 1 from <code>temp_center</code> that are still in <code>nodes_set</code> (unvisited). In an RGG, $d=1$ is identical to Euclidean distance $\leq r$. Append <code>temp_center</code> itself if not already present. <pre>nodes_within = [i for i, d in enumerate(dist_matrix[temp_center]) if i in node_set and d == 1] if temp_center not in nodes_within: nodes_within.append(temp_center)</pre>
3	Handle isolated node If <code>nodes_within</code> is empty (<code>temp_center</code> has no unvisited neighbours), add <code>temp_center</code> directly to the resolving set and remove it from <code>nodes_set</code> . This handles isolated nodes and disconnected components. <pre>if not nodes_within: node_set.remove(temp_center) return {temp_center}, 1, node_set</pre>
4	Initialise heuristic Create a fresh UnitCircle centred on <code>temp_center</code> 's actual (x, y) coordinate with radius r . Each call to <code>uc.add()</code> advances the rectangle-bisection state and returns the next probe point. <pre>x_center, y_center = pos[temp_center] uc = UnitCircle(x_center, y_center, r)</pre>
5	Landmark selection loop For each remaining node, call <code>uc.add()</code> to get the next probe point. Find the k_{nearest} nodes to the probe, pick the highest-entropy one as a landmark, and add it to the local resolving set. <pre>ideal_point = uc.add() candidates = sorted(nodes_remaining, key=lambda v: euclidean_distance(pos[v], ideal_point))[:k_nearest] best = max(candidates, key=lambda c: entropy_of_landmark_candidate(...))</pre>
6	Early exit and merge

After each addition, check if remaining nodes are fully distinguished by their distance signatures to the current set. If so, break. The local resolving set is merged into the global set at the end of the outer iteration.

```
if len(signatures) == len(set(signatures.values())): break
resolving_set.update(temp_set)
```

7 Global resolving check and prune

The outer loop checks `checkIfResolvingSet_igraph` before each iteration and exits when the global set is sufficient. After `max_iters` or full resolution, Zobrist pruning removes redundant landmarks.

```
if checkIfResolvingSet_igraph(g, resolving_set, dist_matrix): break
resolving_set = prune_resolving_set_zobrist_fast(g, resolving_set,
dist_matrix)
```

4.5 Notes and Caveats

- No fallback: `UnitSquare.add()` can theoretically return `None` if the priority queue empties, but in practice the square subdivision is infinite (rectangles always get smaller). For very large node counts this should still be monitored.
- The outer loop has a `max_iters=1000` cap. For very large sparse graphs this could be hit before all nodes are visited. Increasing `max_iters` or adding a `nodes_set` exhaustion check addresses this.
- The `UnitCircle` coordinate mapping is approximate — it maps to the horizontal and vertical diameters only, not uniformly across the full circle. The practical effect is that probe points cluster near the axes through the centre.

Algorithm 3 — Zoomed Square Method

get_metric_dimension_of_graph_with_pruning_igraph_zoomed_square

5.1 Overview

Entry point	<code>get_metric_dimension_of_graph_with_pruning_igraph_zoomed_square(G, r, k_nearest=1, max_iters=1000)</code>
Inner function	<code>get_metric_dimension_of_unit_square_zoomed_square(g, r, temp_center, dist_matrix, nodes_set, k_nearest)</code>
Heuristic	UnitSquareNew — same two-parabola generator as the Grid Method
Neighbourhood	Graph distance $d==1$ (direct neighbours, identical to Euclidean $\leq r$ in RGG)
Centre selection	Degree-weighted random sampling — same as Circle Method
$k_nearest$	Default 1

5.2 Core Idea

The Zoomed Square Method is a hybrid. It borrows the adaptive centre-selection strategy from the Circle Method (degree-weighted random node sampling) and replaces the UnitCircle heuristic with the UnitSquareNew parabolic-curve generator from the Grid Method. The result is a method that adapts to graph density while using the more geometrically principled square-based probe pattern.

Conceptually: instead of imposing a fixed grid, it dynamically 'zooms in' on each selected centre node and treats the surrounding $2r \times 2r$ region as a unit square, applying the same heuristic that was empirically optimised for that geometry.

5.3 Relationship to the Other Methods

Grid Method	Zoomed Square	Circle Method
Fixed grid traversal	Adaptive (degree-weighted)	Adaptive (degree-weighted)
UnitSquareNew (parabolas)	UnitSquareNew (parabolas)	UnitSquare + UnitCircle
Geometric cell membership	Graph $d==1$ (neighbours)	Graph $d==1$ (neighbours)

The Zoomed Square row is highlighted — it inherits the best trait of each parent method.

5.4 Step-by-Step

1	Select centre node (same as Circle Method) Degree-weighted random selection from <code>nodes_set</code>. Falls back to uniform random if all remaining nodes have degree 0. <pre>degrees = [g.degree(n) for n in nodes_set] temp_center = random.choices(list(nodes_set), weights=degrees, k=1)[0]</pre>
2	Collect neighbourhood (same as Circle Method) Gather all unvisited nodes at graph distance 1 from <code>temp_center</code>. Append <code>temp_center</code> itself. Handle isolated nodes by returning <code>{temp_center}</code> immediately. <pre>nodes_within = [i for i, d in enumerate(dist_matrix[temp_center]) if i in nodes_set and d == 1]</pre>
3	Initialise UnitSquareNew generator Create a generator centred on <code>temp_center</code>'s actual coordinates (<code>x_center</code>, <code>y_center</code>) with half-side <code>r</code> and step size <code>d = 2r</code>. Note: with <code>d = 2r</code>, only the two centre probe points will be in-bounds; the rest are handled by the entropy fallback. <pre>x_center, y_center = pos[temp_center] us = UnitSquareNew() d = 2 * r gen = us.point_generator(x_center, y_center, r, d)</pre>
4	Heuristic-guided selection with entropy fallback Same inner loop structure as the Grid Method: probe point from generator → <code>k_nearest</code> candidates → max entropy pick. When generator yields <code>None</code>, fall back to global entropy-greedy over all remaining nodes. <pre>ideal_point = next(gen, None) # if ideal_point is None: entropy-greedy fallback</pre>
5	Early exit, merge, global prune Identical to the other methods: distinct-signatures check for early exit, local set merged into global, Zobrist pruning after the outer loop completes. <pre>resolving_set.update(temp_set) resolving_set = prune_resolving_set_zobrist_fast(g, resolving_set, dist_matrix)</pre>

6. Method Comparison

6.1 Property Table

Property	Grid Method	Circle Method	Zoomed Square Method
Neighbourhood geometry	Axis-aligned square cell (side = $2r$)	Circle of radius r around centre node	Square of side $2r$ around centre node
Centre selection	Fixed grid — row-major traversal	Degree-weighted random sampling	Degree-weighted random sampling
Probe point source	UnitSquareNew curves (two parabolas)	UnitSquare + UnitCircle coordinate transform	UnitSquareNew curves (same as Grid)
Step size d	$d = 1 / \text{ceil}(1/2r) (= 2r)$	Driven by largest-rectangle split	$d = 2r$ (only centre points used in practice)
Neighbourhood filter	Geometric: nodes in grid cell	Graph: $d==1$ (direct neighbours)	Graph: $d==1$ (direct neighbours)
Fallback when heuristic ends	Entropy-greedy over remaining nodes	None	Entropy-greedy over remaining nodes
Early exit condition	Distinct distance signatures	Distinct distance signatures	Distinct distance signatures
Local pruning	None (removed)	None (removed)	None (removed)
Global pruning	prune_resolving_set_zobrist_fast	prune_resolving_set_zobrist_fast	prune_resolving_set_zobrist_fast
Known issue at $r=0.1$	Fails for $n=10, 50, 100$ (sparse/disconnected)	Handles correctly	Handles correctly
Time complexity (dominant)	$O(\text{cells} \times k \times n)$	$O(\text{iters} \times k \times n)$	$O(\text{iters} \times k \times n)$

6.2 Empirical Results Summary

Benchmark: 30 combinations of $n \in \{10, 50, 100, 200, 500, 1000\}$ and $r \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$. One RGG per combination. ICH used as reference.

Validity failures	Grid method fails (returns non-resolving set) for $n=10, 50, 100$ at $r=0.1$ due to disconnected graph + limited generator reach. Circle and Zoomed Square always return valid sets.
Resolving set size	All three methods produce sets within 0-15% of ICH across most parameter combinations. Grid method tends to be competitive or better at moderate r . Circle method overshoots at large r (e.g. $n=1000, r=0.7$: Grid=182, Circle=254, ICH=166).
Speed	Grid method is fastest at small-to-medium n (fixed traversal, no random restarts). Circle and Zoomed Square are faster at large n/r due to early global exit. ICH is 5-50x slower than all three heuristics at large n .

At $n=500$, $r=0.1$

Grid=63, Circle=65, Square=65, ICH=69 — all heuristics beat ICH on set size, which is unusual and likely reflects a single disconnected RGG instance.

6.3 Recommended Use

- For well-connected graphs ($r \geq 0.3$ or $n \geq 200$): all three methods are reliable. Grid method is fastest and produces the smallest sets at moderate density.
- For sparse/small graphs ($r = 0.1$, $n < 200$): use Circle or Zoomed Square. Grid method is unreliable in this regime.
- For thesis/publication: run all three and report the minimum, using ICH as ground truth for validation.

7. Open Issues and Future Work

Grid d scaling	$d = 1/\max(m,n) = 2r$ leaves the generator with almost no reach per cell. Fix: $d = 2r / \max(\text{len}(\text{nodes_within}), 1)$. Expected to fix the $r=0.1$ validity failures.
Zoomed Square d	$d = 2r$ means only the two centre points are ever used. Same fix applies: scale d by node density or use $d = r/2$.
Circle fallback	<code>uc.add()</code> has no <code>None</code> guard. If <code>UnitSquare</code> 's priority queue ever empties (theoretically impossible for infinite subdivision but worth guarding), the algorithm will crash with a <code>TypeError</code> .
k_nearest > 1	With $k_{\text{nearest}}=1$ the heuristic fully determines picks and entropy is irrelevant. Testing $k_{\text{nearest}}=3$ or 5 would reveal whether the entropy criterion adds value when given a candidate pool to choose from.
Entropy-only baseline	A version using only entropy-greedy with no heuristic probe points would confirm or refute whether the geometric heuristics actually improve on pure information-theoretic selection.
Disconnected graphs	None of the methods handle disconnected RGGs robustly. A pre-check that adds one node per connected component to the resolving set before running the main algorithm would fix this cleanly.
Visualisation	A matplotlib step-through or animation showing the grid/neighbourhood, probe points, and selected landmarks per iteration would aid debugging and presentation.